



Linux Simplified

Personal Study Notes

✍ Handwritten by: **Akash Francis**

* Shell Types *

Shell

Shell → `echo $shell`

- ↳ Korn Shell `<ksh>`
- ↳ Bourne Shell `<sh/shell>`
- ↳ C Shell `<csh/tsh>`
- ↳ Z Shell `<zsh>`
- ↳ Bourne Again Shell `<bash>`

* Commands - Directories *

- `mkdir -p /d1/d2/d3` ← creating directory tree
- `rm -r /d1/d2/d3` ← removing directory recursively
- `cp -r d1/d2/d3` ← copy directory recursively

* Commands - Files *

- `touch file-name.txt` ← create a file (empty)
- `cat > file-name.txt` ← add content to file
- `cat file-name.txt` ← view content of the file

* Working with Shell *

- (i) `echo` → print
- (ii) Home directory → `~` (tilde)
- (iii) Syntax: `command <option> argument`
Ex: `echo -n Hello`
- (iv) `uptime`

* Command Types *

In Built-in Commands

`echo` / `cd` / `pwd` / `set`

External Commands

`mv` / `date` / `uptime`

💡 Use the `type` command to know the type of any command!

* Paths *

Absolute Path

Path from root directory (starts with `/`)

Relative Path

Path relative to current directory

`.` → current directory

`..` → parent directory

* Navigation Commands (LIFO) *

pushd

`pushd [directory]` - add to top of stack & make current

`pushd -n [dir]` - add at 2nd position without changing current dir

`pushd +N / -N` - move to directory at position N

`dirs -l -v` - display directory stack

popd

`popd` - removes top directory

`popd -n` - removes 2nd top directory

`popd +N / -N` - removal from any position

✦ Renaming a file = moving file into same directory by renaming it

* Pagers *

Displays content of file in user-friendly way — view info one page at a time

more

`more file-name.txt`

less

```
less file-name.txt
```

* Long List (ls) *

- `ls -l` - long list
- `ls -a` - list all files including hidden
- `ls -lt` - long list in order created
- `ls -ltr` - long list in reverse order created

* Commands to Get Help *

- `whatis [command]`
- `man [command]` → user manual for command/program/file
- `[command] --help`
- `apropos [command] ≈ man -k [command]` → when you know the action but not the exact command

* chsh Command *

→ Change user's login shell (only superuser can change)

Syntax: `chsh [option] [login]`

- `-l` → specify login shell
- `-u` → print list of shells
- `-v` → show info about version & exit
- `-s` → set shell as login shell

Example: `chsh -s /bin/sh bob`

* Features of Bash Shell *

- ① Bash Auto-completion (use `Tab`)
- ② Setting Alias → nickname for particular command
e.g. `alias dt=date`
- ③ Searching History using `history` command

* Bash Environment Variables *

Stores info about how Linux system works — e.g. secrets & configuration data/location

- ① Display value of env variable → `echo $SHELL`
- ② Display list of global env variables → `env`

`set` → displays both global & local env variables

- ③ Setting Global Environment Variable:

`export Env-Variable=value` OR `set Env-Variable=value`

- ④ Setting Local Environment Variable: `Env-Variable=value`

- ⑤ User-wide environment variables (personalized) configured in:

`~/.bashrc` / `~/.profile` / `~/.bash_login` / `~/.bash_profile`

- ⑥ System-wide environment variables (accessible to all users):

`/etc/environment` / `/etc/profile` / `/etc/profile.d` / `/etc/bash.bashrc`

- ⑦ To unset: `unset Env-Variable-Name` OR `Env-Variable-Name= ''`

* PATH Variable *

Contains list of all directories the system searches when any command is run. Helps user run programs from any directory without typing the full path.

Temporary

Useful for current session only

`export PATH=path-value`

Permanent

Useful any time after reboot

S1: `echo $0` (check shell)

S2: Edit shell RC file

S3: `export PATH=path-value`

S4: `source ~/.bashrc`

S5: `echo $PATH`

* Customizing Bash Prompt *

Default prompt: `username@hostname:~$`

- `$` → standard account `#` → root account
- **PS1** → customizable → `[\u@\h \W]\$`
- **PS2** → used when user provides long command separated by `\` character (`>`)
- **PS3** → to select command (`+`)
- **PS4** → to run shell script in debug mode

Bash prompt customized from `~/.bashrc` file

Steps: ① Backup bashrc ② Edit PS1 value ③ Run `source ~/.bashrc`

Temporary: `export PS1="name"` or `PS1="name"`

* Linux Kernel *

Acts as the middleman between applications and hardware (like a librarian between students and resources).

- ① **Memory Management** → distribute memory to various processes running on the system
- ② **Process Management** → distribute processes to CPU
- ③ **Device Drivers** → provides OS with a software interface to allow specific device to function
- ④ **System Calls & Security** → requests made by applications to kernel to perform specific tasks
- ⑤ **Monolithic** → entire OS is running in kernel space
- ⑥ **Modular** → allows part of the system to be added/removed without affecting the entire system

* Kernel Version *

Example: `4.15.0-72-generic`

- `4` → kernel version
- `15` → major version (identify device driver associated with device)
- `0` → minor version (identify instances of devices attached to major number)
- `72` → patch release

`uname` → print OS name `uname -r` → print kernel version

* Kernel Space vs User Space *

Kernel Space

- Part of memory where core OS & kernel reside
- Full privileges & access
- Handles system call requests through system call interface

User Space

- Where user applications & processes run
- Limited privileges & rights
- System calls invoked here to request services from OS

✧ Kernel space & user space interact through system calls

* Hardware Info Commands *

Command	Purpose
<code>dmesg</code>	Displays kernel ring buffer messages (hardware, device drivers, modules)
<code>udevadm monitor</code>	Prints received events for UDEV and KERNEL
<code>lspci</code>	Lists PCI devices & buses connected to the system
<code>lsblk</code>	Lists block devices (HDD, SSD, storage)
<code>lscpu</code>	Provides details about CPU-related components
<code>lsmem --summary</code>	Memory summary
<code>free -m/g</code>	Free memory in MB/GB
<code>lshw</code>	Lists all hardware info

Core → individual processing unit | **Thread** → smallest sequence of programmed instruction | **Socket** → physical interface where CPU is installed

No. of Threads = No. of Sockets × No. of Cores/CPU × No. of Threads/CPU

* Swap Memory *

Memory available on HDD/SSD which contains inactive data from RAM (acts as overflow for RAM)

* Linux Boot Sequence *

- 1 **BIOS** → integrity checks for HDD/SSD → Boot loader loaded from Master Boot Record (MBR) → MBR is in 1st sector of HDD/SSD (`/dev/hda` or `/dev/sda`) → OS loaded to main memory
- 2 **Boot Loader (GRUB2)** → Grand Unified Boot Loader (new systems) | LILO (old systems) → GRUB loads kernel into memory
- 3 **Kernel Initialization** → mounts root file system (grub.cfg) → `/sbin/init` program executed (PID=1) → Temporary file system established using initrd until permanent is established
- 4 **INIT Process (systemd)** → decides runlevel using init file at `/etc/inittab` → executes runlevel programs → all services started → system ready!

Runlevel	Systemd Target	Function
5	<code>graphical.target</code>	Boots into Graphical Interface
3	<code>multiuser.target</code>	Boots into Command Line Interface

* Linux File Types *

- ① **Regular Files** → e.g. image, text file, etc.
- ② **Directory**
- ③ **Special Files**
 - **3.1 Character File** → represents I/O devices under `/dev` directory, allows OS to communicate with device
 - **3.2 Block Files** → represents block devices like HDD/SSD
 - **3.3 Links** → connection between filename & actual data on disk
 - **3.4 Socket Files** → communication between processes; uses IP address + port No.; used for asynchronous communication (time gap)
 - **3.5 Named Files (FIFO)** → interprocess communication within a system; synchronous communication (no time gap)

Hard Link

```
ln file file-HD-link
```

Both files share same data & inode number.
If either is deleted, the other is NOT affected.

Soft Link (Symbolic)

```
ln -s file file-softlink
```

Different inode numbers — pointer/shortcut.
If link is modified/deleted, other will dangle.

`inode` → unique identifier for a file that contains metadata about the file → use `ls -li`

* Filesystem Hierarchy *

Directory	Purpose
<code>/opt</code>	Standard directory to install non-OS related software
<code>/mnt</code>	Directory for temporary mounting of external file system (manual mount)
<code>/media</code>	Automatic mount directory for external files
<code>/dev</code>	Represents devices attached to the system through device files; gives access of peripheral devices to the OS
<code>/bin</code>	Contains executable binary files for basic commands
<code>/tmp</code>	Stores data needed for short period of time
<code>/etc</code>	Contains configuration files
<code>/lib</code>	All helpful libraries used by the system
<code>/home</code>	User-specific files & directories
<code>/usr/bin</code>	Accessible to all users; has system-wide commands
<code>/usr/sbin</code>	Accessible to system administrators (root) only

* Package Manager *

A software tool that manages: **Installation, Upgradation, Removal** of packages (S/W binaries, metadata, config files)

.DEB (Ubuntu/Debian/Linux Mint)

DPKG → manual (low-level) dependency handling

APT → automatic (high-level), user-friendly

APT-GET → low level backend

.RPM (RHEL/CentOS/Fedora)

RPM → manual (low-level) dependency handling

YUM → automatic (high-level)

DNF → fast & efficient

*** Working with YUM ***

- `yum install package-name`
- `yum repolist`
- `yum provides scp`
- `yum remove package-name`
- `yum update package-name`

*** Working with APT / APT-GET ***

- `apt install package-name`
- `apt update`
- `apt upgrade`
- `apt edit-sources`
- `apt remove package-name`
- `apt search package-name`
- `apt list`

*** Working with RPM & DPKG ***

Action	RPM	DPKG
Install	<code>rpm -ivh telnet.rpm</code>	<code>dpkg -i telnet.deb</code>
Uninstall	<code>rpm -e telnet.rpm</code>	<code>dpkg -r telnet.deb</code>
Upgrade	<code>rpm -Uvh telnet.rpm</code>	<code>dpkg -i telnet.deb</code>
Query	<code>rpm -q telnet.rpm</code>	<code>dpkg -l telnet</code>
Verify	<code>rpm -Vf <path></code>	<code>dpkg -p <path></code>

*** SCP - Secure Copy Protocol ***

Transferring data securely over SSH

- **Pre-requisites:** SSH installation + root user access for both client & server
- **Syntax:** `scp [OPTIONS] [[user@]src_host:]file1 [[user@]dest_host:]file2`
- `-P` → port to connect to remote machine

Use Cases

- ① Copy files within same machine
- ② Local → Remote
- ③ Remote → Remote

Viewing File Sizes

```
du -sk file.img
du -sh file.img
ls -lh file.img
```

* Archiving Files (tar) *

- `tar -cf test.tar file1 file2 file3` → create new archive (`c` =create, `f` =specify name)
- `tar -tf test.tar` → list files in archive
- `tar -xf test.tar` → extract files in archive
- `tar -zcf test.tar file1 file2 file3` → create compressed archive

* Compressing Files *

Compression depends on: compressed data + compression algorithm + compression level

Compress	Decompress	View without decompressing
<code>bzip2 file</code>	<code>bunzip2 file.bz2</code>	<code>zcat</code> / <code>bzcat</code> / <code>xzcat</code>
<code>gzip file</code>	<code>gunzip file.gz</code>	
<code>xz file</code>	<code>unxz file.xz</code>	

* Searching for Files & Directories *

- `locate City.txt` → quick search using database
- `updatedb` → update locate database
- `find /home/michael -name City.txt` → real-time search

* GREP - Searching Pattern in a File *

- `grep second sample.txt` → find pattern
- `grep -i capital sample.txt` → case insensitive
- `grep -r "third line" /home/michael` → recursive search
- `grep -v "printed" sample.txt` → invert match
- `grep -w exam examples.txt` → whole word match

- `grep -A1 Arsenal premier-league.txt` → lines after match
- `grep -B1 4 premier-league.txt` → lines before match

* I/O Redirection *

- ① **Redirect Stdout:** `>` → overwrites output to target/new file
- `>>` → appends output to target file
- ② **Redirect Stderr:** `2>` → overwrites error output to target/new file
- `2>>` → appends error output to target file
- ③ **Discard Output:** `echo "Akash" > /dev/null` (directory is always empty, hence dumps data)
- ④ **Redirect Stdin:** `cat < data.txt` → command takes text file as input

* Command Line Pipes *

Connect output of one command as input to another: `command1 | command2`

Example: `grep Hello sample.txt | less`

`tee` → redirect output to file + display in prompt (override)

`tee -a` → append output to file + display output

* Text Editors in Linux *

Small Editing

`cat > file` → override text

`cat >> file` → append text

(till Ctrl+D)

Large Editing

`vi` editor

`vim` editor (vi improved)

* VI Editor Modes *

Mode	Enter	Actions
Command Mode	Default on open	Move, copy (yy), paste (p), delete letter (x), delete line (dd), undo (u), redo (r), find (/line or ?line)
Insert Mode	<code>i</code> , <code>I</code> , <code>o</code> , <code>O</code> , <code>a</code> , <code>A</code>	Edit file, add/remove text
Last Line Mode	<code>:</code> from command mode	<code>:w</code> save, <code>:q</code> quit, <code>:wq</code> save & quit, <code>:q!</code> force quit

VIM = VI Improved → adds: Completion, Spell Check, Compare on Merging, GUI, Plugins, Syntax Highlighting